DYNAMIC LEARNING

PERSONALIZED LEARNING-RESOURCE RECOMMENDATION SYSTEM USING LEARNER PROGRESS AND ADAPTIVE REVISION

A PROJECT REPORT

Submitted by

NOVIN KUMAR M (310123104068)

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

ANAND INSTITUTE OF HIGHER TECHNOLOGY

(An Autonomous Institution)

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

ANAND INSTITUTE OF HIGHER TECHNOLOGY

(An Autonomous Institution)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this project "DYNAMIC LEARNING — PERSONALIZED LEARNING-RESOURCE RECOMMENDATION SYSTEM USING LEARNER PROGRESS AND ADAPTIVE REVISION" is the bonafide work of NOVIN KUMAR M (310123104068) who carried out the project work under the guidance and supervision of the department.

SIGNATURE

Mrs. M. Maheshwari, M.E., (Ph.D.)
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Anand Institute of Higher Technology

SIGNATURE

Mrs. M. Maheshwari, M.E., (Ph.D.)
SUPERVISOR
Department of Computer Science and Engineering
Anand Institute of Higher Technology

Submitted for University Examination held in October 2026

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I express my sincere gratitude to the management and faculty of Anand Institute of Higher Technology for providing the facilities and encouragement required to complete this project.

I extend my heartfelt thanks to Mrs. M. Maheshwari, M.E., (Ph.D.), Head of the Department of Computer Science and Engineering, for her support and guidance throughout the project work.

I also thank the teaching and non-teaching staff members of the Department of Computer Science and Engineering for their valuable assistance, suggestions, and encouragement.

Finally, I thank my family and friends for their continuous support and motivation during the development, testing, and documentation of Dynamic Learning.

ABSTRACT

Dynamic Learning is a lightweight, explainable Flask web application for recommending a next learning resource from a small Python learning catalogue. The application combines server-scored quiz attempts, topic-level mastery estimates, prerequisite metadata, resource-completion records, and a time-based revision policy. It presents a primary recommendation and supporting resources together with human-readable reasons, so that a learner can inspect the evidence behind the displayed next step. The seeded catalogue contains seven Python topics—Variables, Data Types, Conditionals, Loops, Functions, Lists, and Basic Algorithms—along with 21 learning resources and seven quizzes. Working multi-question quizzes are provided for Variables, Conditionals, Loops, and Functions.

The learner model uses a transparent exponential moving average (EMA). With the current starting rate of 0.35, each new quiz score contributes 35% of the update while the previous estimate contributes 65%. Candidate resources are filtered using completion and revision timing, prerequisite readiness is represented explicitly, and a deterministic priority score is used to order the remaining candidates. The application persists users, quiz attempts, question responses, resource completions, mastery scores, prerequisites, and resource interactions through SQLAlchemy models, with SQLite used in the local development workflow.

The project is an academic prototype and not a production learning platform. Its offline comparison uses four hand-built synthetic cases and compares an adaptive learner-signal ranker with a stable catalogue-order baseline and a content-based baseline. The regenerated benchmark reports ranking metrics only; it is not a learner study and does not establish educational improvement, adaptive-method superiority, accurate forgetting detection, novelty, or reproduction of the supplied base paper. The final audit recorded 167 passing tests with warnings treated as errors, successful Python compilation and JavaScript syntax checks, dependency verification, security checks, and an HTTP persistence workflow. A manual browser walkthrough remains necessary for visual, responsive, keyboard, and browser-JavaScript validation.

TABLE OF CONTENTS

ACKNOWLEDGEMENT ii
ABSTRACT iii
TABLE OF CONTENTS iv
LIST OF FIGURES vi
LIST OF TABLES vii
Chapter 1 — Introduction 1
1.1 Background 1
1.2 Problem statement 1
1.3 Objective 1
1.4 Scope 2
1.5 Report organisation 2
Chapter 2 — Literature Survey 3
2.1 Purpose and verification boundary 3
2.2 Adaptive user models and recommender systems for learning 3
2.3 Student modelling, knowledge tracing, and revision 3
2.4 The supplied base paper and project relationship 3
2.5 Scoped implementation question 4
Chapter 3 — Analysis 5
3.1 System analysis 5
3.1.1 Problem identification 5
3.1.2 Existing-system context 5
3.1.3 Proposed system 5
3.2 Requirement analysis 5
3.2.1 Functional requirements 5
3.2.2 Non-functional requirements 6
3.2.3 Hardware specification 7
3.2.4 Software specification 7
Chapter 4 — Design 8
4.1 Overall design 8
4.2 Learner workflow 8
4.3 Use-case design 9
4.4 Class and data model 10
4.5 Quiz-submission activity 11
4.6 Recommendation sequence 12
Chapter 5 — Implementation 14

5.1 Modules and responsibilities 14
5.2 Data collection and server-side grading 14
5.3 Mastery update 14
5.4 Candidate filtering and ranking 15
5.5 Completion and revision persistence 15
5.6 Explainability and UI contract 15
Chapter 6 — Testing 16
6.1 Verification strategy 16
6.2 Representative test plan and evidence 16
6.3 Security and operational boundaries 17
6.4 Browser and packaging boundary 17
Chapter 7 — Results and Discussion 19
7.1 Application results 19
7.2 Exact synthetic offline benchmark 19
7.3 Benchmark metric definitions and validity limits 20
7.4 Learning-rate sensitivity 20
Chapter 8 — User Manual 22
8.1 Local setup 22
8.2 Learner workflow 22
8.3 Administrator evaluation 22
8.4 Troubleshooting 22
Chapter 9 — Conclusion 24
Chapter 10 — Future Enhancement 24
APPENDIX 1 — SUPPLIED BASE PAPER 25
APPENDIX 2 — SCREENSHOTS 26
APPENDIX 3 — INTERNSHIP CERTIFICATE 33
APPENDIX 4 — EVIDENCE AND DISCLOSURE NOTE 34
REFERENCES 35

LIST OF FIGURES

Figure 4.1 Proposed system architecture

Figure 4.2 Learner workflow and recommendation loop

Figure 4.3 Use-case diagram

Figure 4.4 Core data model

Figure 4.5 Quiz submission and adaptation activity

Figure 4.6 Recommendation-generation sequence

LIST OF TABLES

Table 3.1 Functional requirements

Table 3.2 Non-functional requirements

Table 5.1 Modules and responsibilities

Table 6.1 Representative test plan and evidence

Table 7.1 Synthetic offline benchmark results

Table 7.2 Per-scenario synthetic results

Table 7.3 Learning-rate sensitivity formula results

Table 7.4 Learning-rate sensitivity replay

Chapter 1 — Introduction

1.1 Background

Learning resources are often available as a catalogue, but a catalogue alone does not identify which topic a learner should study next. A learner may have different levels of evidence across topics, may need a prerequisite before attempting a more advanced resource, or may benefit from revisiting a resource after a period of time. Dynamic Learning addresses this implementation problem with a small local application that records learner interactions and turns them into inspectable recommendations.

The project uses a seven-topic Python path: Variables, Data Types, Conditionals, Loops, Functions, Lists, and Basic Algorithms. The path contains seeded resources and explicit prerequisite links. Quiz activity is recorded by topic, and the application separates a topic that has not yet been assessed from a topic whose current estimate is low. This distinction prevents an empty history from being represented as fabricated evidence of weakness.

The design deliberately favours deterministic rules over a paid API, cloud model, GPU, or opaque machine-learning service. This makes the local demonstration reproducible and permits the recommendation reason, mastery estimate, candidate filtering, and priority calculation to be inspected.

1.2 Problem statement

A static list of learning resources cannot, by itself, express whether a learner has demonstrated difficulty with a particular topic. A single aggregate progress value can also hide topic-specific differences. In addition, an advanced resource may be inappropriate when a prerequisite topic has not reached the stored minimum mastery, and a recently completed resource may not be a useful immediate recommendation. Finally, a recommendation is difficult to audit if the learner cannot see why the item was selected.

The problem addressed in this project is therefore the construction of a small, persistent, and explainable learning-resource recommendation flow that:

1. records quiz and completion evidence for an authenticated learner;
2. maintains topic-level mastery estimates without presenting them as grades or ground-truth knowledge;
3. represents prerequisite relationships explicitly;
4. filters recent completions and allows older completed resources to return as revision candidates; and
5. produces a deterministic queue with human-readable reasons.

1.3 Objective

The main objective is to improve the personalization of learning-resource recommendations by using the learner's recorded progress, topic-level evidence, prerequisite readiness, and revision timing. The specific objectives are to:

1. build a learner account and progress dashboard with session-bound access;
2. provide the seven-topic Python learning path;
3. provide working quizzes for Variables, Conditionals, Loops, and Functions;
4. calculate topic-wise mastery from server-scored quiz attempts;
5. record quiz history, question responses, resource completion, and resource interactions;
6. recommend resources with a deterministic priority and a human-readable explanation; and
7. compare the adaptive ranker with stable-catalog and content-based baselines without inventing learner outcomes.

1.4 Scope

The implemented scope includes registration and sign-in, a learner dashboard, a learning library with filtering, topic quizzes, server-side grading, feedback, topic mastery and history, resource completion, prerequisite-aware recommendation logic, revision eligibility, and an admin-only offline evaluation view. The application stores its state using SQLAlchemy models and is demonstrated locally with SQLite.

The scope does not include a production content marketplace, collaborative filtering, a real learner study, a validated psychological threshold, or a validated forgetting model. The seeded external resource URLs are learning links used by the demonstration; they are not hosted by the application and may change or become unavailable. The audit did not certify production deployment, multi-worker concurrency, load behaviour, TLS deployment, dependency vulnerability status, or long-running operational behaviour. The final audit also did not use a browser rendering engine, so visual layout, responsive behaviour, keyboard behaviour, and JavaScript click execution require a separate manual walkthrough.

1.5 Report organisation

Chapter 2 records the verified literature map and the careful relationship between the project and the supplied base paper. Chapter 3 analyses the problem and states requirements. Chapter 4 describes the architecture, workflow, use cases, data model, activity, and sequence of operations. Chapter 5 explains the implementation and formulas. Chapter 6 records testing and audit evidence. Chapter 7 discusses application results, the synthetic benchmark, and the learning-rate sensitivity check. Chapter 8 provides the user manual. Chapters 9 and 10 state the conclusion and future enhancements. The appendices preserve evidence and packaging notes, followed by the references.

Chapter 2 — Literature Survey

2.1 Purpose and verification boundary

The literature map in this report is a verified identity-and-scope summary rather than a systematic literature review. The project guide records source links and distinguishes publication identity from full-text limitation evidence. The audit verified the identities of the listed works using DOI, Crossref, ACL, or NeurIPS records as applicable. Only the abstract of *Deep Knowledge Tracing* was substantively readable during the audit among the research summaries used for the project. Publisher access challenges, rate limits, and unreadable PDF extraction are verification limits; they are not evidence that a work lacks a capability.

2.2 Adaptive user models and recommender systems for learning

Brusilovsky and Millán's chapter, "User Models for Adaptive Hypermedia and Adaptive Educational Systems," appears in *The Adaptive Web*, pages 3–53. The chapter identity, authors, pages, and the parent volume's 2007 year were verified through DOI and Crossref records. The project guide records that the full Springer text was blocked during verification. Accordingly, earlier attributions of particular cold-start or sparse-evidence limitations are not presented here as claims established by that chapter.

Manouselis, Drachsler, Verbert, and Duval's *Recommender Systems for Learning* is a SpringerBriefs book with a 2013 print year in the verified Crossref record. It is distinct from a different 2011 technology-enhanced-learning chapter. The book's identity and year were verified, while full-text limitation claims were not passage-verified. Its relevance to this project is the general problem of recommendation in learning environments, not a claim that the book establishes a particular missing capability.

2.3 Student modelling, knowledge tracing, and revision

Piech and colleagues' "Deep Knowledge Tracing," published in *Advances in Neural Information Processing Systems 28* in 2015, is described in the verified proceedings abstract as using recurrent neural networks to model student interactions and to perform predictive evaluation on real student data. The project does not reproduce that method. Instead, Dynamic Learning uses a transparent topic-level EMA so that the calculation can be inspected in a small local demonstration. The project guide specifically removes an unsupported attribution that the cited work is less interpretable; the abstract itself claims straightforward interpretation and discovery of curriculum structure.

Settles and Meeder's "A Trainable Spaced Repetition Model for Language Learning" was verified as an ACL 2016 paper, pages 1848–1858, with a corresponding DOI record. The stated scope is language-learning spaced repetition. The PDF endpoint used during the audit did not provide readable text, so detailed model limitations or transfer claims are not attributed to the authors. Dynamic Learning uses a conservative 14-day completion policy; this is a local application rule, not a validated forgetting detector or a claim about the cited model.

2.4 The supplied base paper and project relationship

The supplied base paper is Hongnan You, Lijun Xu, and Dongping Wu, "MFF-CF: A Multi-Feature Fused Collaborative Filtering Framework for Personalized Learning Resource Recommendation in Smart Education," *Discover Artificial Intelligence*, volume 6, article 860, DOI 10.1007/s44163-026-01715-0. Crossref metadata verified the title, authors, journal, DOI, and online date of 21 August 2026. The full Springer article could not be read during the audit because the publisher returned a client challenge.

Dynamic Learning is not a reproduction of MFF-CF and does not implement collaborative filtering. The project uses a deterministic content- and learner-signal-based ranker over a small seeded catalogue.

Statements about GPU requirements, compute burden, weak-topic identification, or revision/prerequisite modelling in the base paper are not made because the full text was not verified. The benchmark in this report therefore does not compare Dynamic Learning against MFF-CF and cannot support superiority or a reproduction claim.

2.5 Scoped implementation question

The project investigates a narrower implementation and evaluation question: how can topic-level quiz evidence, explicit prerequisites, completion and revision rules, and recommendation reasons be made visible and reproducible in a small local learner flow when collaborative interaction data is unavailable? This is a scope question, not a proven research gap or novelty claim. The report does not assert that prior work lacks these ideas, and no systematic comparative reproduction was performed.

Chapter 3 — Analysis

3.1 System analysis

3.1.1 Problem identification

The analysis begins with the learner's next-step decision. A generic catalogue may offer resources but does not necessarily identify a topic-specific difficulty. Aggregate progress can conceal a weak topic, and a recommendation can be difficult to trust when its reason is invisible. A resource may also depend on another topic, while a resource completed very recently may not be an immediate next step.

Dynamic Learning makes the evidence path explicit. Quiz attempts are stored with their score and answer data. Individual question responses are stored with correctness. A topic-level score is maintained separately from the raw attempt history. Completions are stored separately and used by the candidate filter. Prerequisites connect topics and include a minimum mastery value. The resulting recommendation is an estimate derived from these records, not a statement of the learner's definite knowledge.

3.1.2 Existing-system context

The project guide uses static catalogues, manual resource selection, aggregate-only progress views, and opaque or data-hungry approaches as context for the design problem. These are contextual categories, not claims about a specific named paper. The audit also warns that limitation claims from the related literature were not fully passage-verified. The analysis therefore focuses on the behaviour implemented by this repository rather than attributing unsupported deficiencies to prior authors.

3.1.3 Proposed system

The proposed system is a Flask application with a responsive learner interface, SQLAlchemy persistence, deterministic domain services, a seeded Python catalogue, and an evaluation view. After registration and sign-in, a learner can view a cold-start dashboard, browse and filter the library, open a quiz, submit answers, inspect per-question feedback, complete a resource, and return to the dashboard. A newly registered learner has no quiz evidence; the interface therefore uses an onboarding-style cold-start queue and displays unassessed topics as **Not assessed** rather than weak.

When a quiz is submitted, the learning route validates the JSON request, grades only exact option values, creates the attempt and question-response records, updates mastery, commits the related changes together, and returns feedback and an adaptation comparison. The next recommendation plan includes a primary resource and up to two supporting resources. A recent completion is filtered from the immediate queue, while an older completion may be eligible again under the revision time policy. Reasons describe the rule-based evidence used by the ranker.

The advantages of this design are inspectability, deterministic replay, a small local resource footprint, and a clear separation between evidence and interpretation. Its boundaries are equally important: the catalogue is demonstration data, mastery is an estimate, the constants are not validated educational settings, the revision policy does not establish forgetting, and the synthetic comparison does not measure learning outcomes.

3.2 Requirement analysis

3.2.1 Functional requirements

Table 3.1: Functional requirements

ID	Functional requirement	Implemented interpretation

FR-01	Registration and sign-in	Create a learner account and use a session-bound authenticated flow.
FR-02	Private learner state	Keep attempts, completions, mastery, and recommendation state associated with the signed-in user.
FR-03	Dashboard	Display the user, topics, assessed mastery, history-derived counts, and recommendation queue.
FR-04	Catalogue browsing	Browse resources and filter by text, topic, and kind in the learning library.
FR-05	Topic quizzes	Load a quiz with its questions and answer options for the selected topic.
FR-06	Server-side grading	Validate the JSON object, answer object, question IDs, and option values on the server; grade exact options.
FR-07	Feedback	Return score, correctness, correct answer, and question explanation for each submitted question.
FR-08	Mastery and history	Store quiz attempts, question responses, and topic mastery estimates.
FR-09	Resource completion	Record a completion and resource interaction, preserving one unique completion per user-resource pair.
FR-10	Recommendation	Filter, rank, and explain a primary recommendation and supporting resources.
FR-11	Prerequisite handling	Represent topic prerequisites and penalise candidates with unmet prerequisite mastery.
FR-12	Revision handling	Suppress recent completions and allow resources completed at least 14 days earlier to return as revision candidates.
FR-13	Evaluation	Expose the offline benchmark only through the admin-protected evaluation flow.
FR-14	Protected writes	Use session and CSRF protections for form and JSON write operations as verified by the audit.

3.2.2 Non-functional requirements

Table 3.2: Non-functional requirements

Requirement	Application interpretation and evidence boundary
Explainability	Recommendation priority and reason are returned with the resource; mastery is described as an estimate.
Reproducibility	The seeded catalogue, deterministic ordering, explicit formulas, and offline cases permit replay.
Correctness	Tests cover input validation, exact-option grading, atomic writes, metrics arithmetic, and candidate consistency.
Security	The audit covers password hashing, sessions, CSRF, access control, production safeguards, and reset guards.

Maintainability	Routes, models, domain services, templates, and evaluation services have separate responsibilities.
Accessibility and responsive behaviour	The README documents semantic landmarks, skip navigation, visible focus, labelled controls, meters, live regions, keyboard-selectable answer cards, reduced-motion support, and narrow-screen breakpoints. These properties still need browser walkthrough confirmation.
Persistence	The HTTP audit verified that attempts, responses, completions, and mastery can be read by a new application instance from the same disposable SQLite database.
Local resource use	The application is designed to run locally with Flask and SQLite; no GPU, cloud service, or LLM is required.
Availability, scalability, and load performance	These remain design considerations. The audit's microsecond-scale ranking timings are not throughput or end-to-end latency tests.

3.2.3 Hardware specification

No measured hardware minimum, deployment capacity, or scalability limit was established by the audit. A practical development machine must be capable of running Python, Flask, the installed dependencies, and a local SQLite database, with sufficient storage for the virtual environment and project files. This is a setup consideration rather than a measured hardware specification. External learning links also require network access when the learner follows them, but the application itself is designed as a local Flask flow.

3.2.4 Software specification

The documented software stack consists of Python, Flask, Flask-SQLAlchemy, SQLAlchemy, Flask-WTF, WTForms, SQLite for local development, HTML/Jinja templates, CSS, vanilla JavaScript, pytest, and a modern browser. The repository provides `requirements.txt`, a seed script, a Flask entry point, and a test suite. A typical local setup is:

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS/Linux
source .venv/bin/activate
pip install -r requirements.txt
python seed.py
flask --app run.py run --debug
```

The README states that the application reads process environment variables rather than automatically loading a `.env` file. A deployment should set a strong random `SECRET_KEY`, use the intended environment settings, and disable the development demonstration account. Private values and published development credentials are not reproduced in this report.

Chapter 4 — Design

4.1 Overall design

Dynamic Learning follows a browser–Flask–service–ORM–database arrangement. The responsive Jinja interface sends authenticated requests to Flask blueprints. Authentication and access-control routes manage sessions. Dashboard, learning, and evaluation routes obtain domain data and call services for mastery, recommendation, catalog, metrics, and offline evaluation. SQLAlchemy models persist learner and catalogue state. SQLite is used for the local development flow, while the documented architecture also shows a PostgreSQL option for the database layer; production use would require explicit deployment and migration controls that were not certified by the audit.

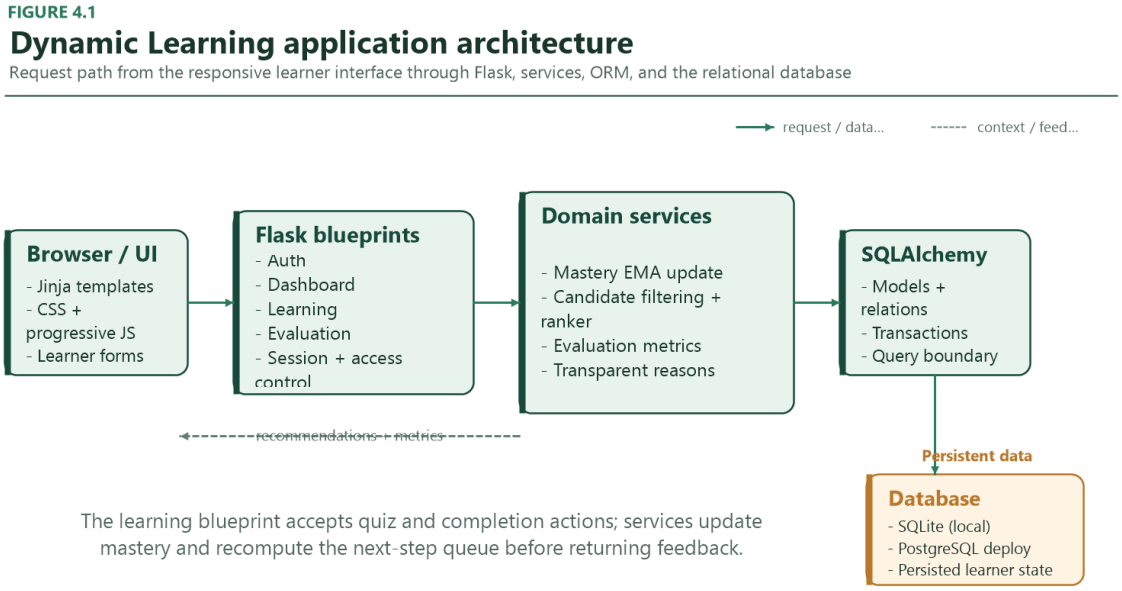


Figure 4.1: Proposed system architecture

The design separates evidence collection from recommendation interpretation. Quiz submission writes an attempt, its question responses, and the refreshed mastery state. Completion writes the unique completion when needed and records a resource interaction. The recommender consumes the persisted state and produces a ranked plan with reasons. The evaluation service compares rankers over a separate synthetic dataset without writing to the learner database.

4.2 Learner workflow

The principal workflow is:

1. The learner registers or signs in.
2. The dashboard displays the cold-start catalogue or the learner's current assessed topics and recommendation plan.
3. The learner browses the library or opens a topic quiz.
4. The quiz route loads questions and options; the submission endpoint validates the request.
5. The server grades exact options and records the attempt and individual responses.
6. The mastery service updates the topic estimate using the EMA rule.

7. The recommendation service filters recently completed resources, checks prerequisites, calculates priorities, and orders candidates deterministically.
8. The response displays score, feedback, updated mastery, recommendation reasons, and an adaptation comparison.
9. The learner can complete a resource; a later request can make an older completion eligible for revision.

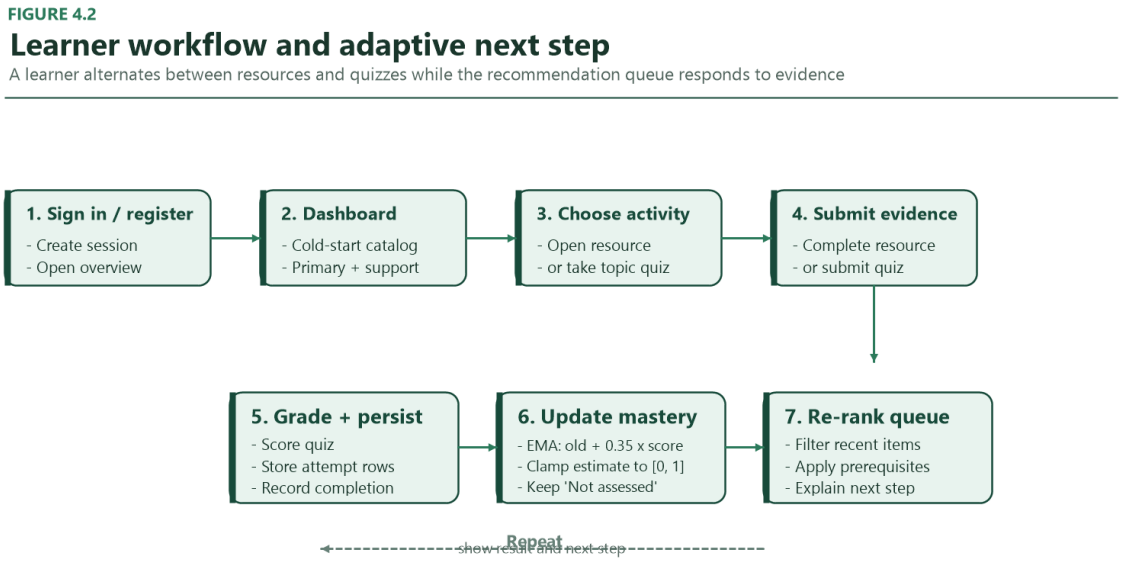


Figure 4.2: Learner workflow and recommendation loop

4.3 Use-case design

The principal actors are the Learner, the Administrator, and the External learning-resource website. The Learner can register, sign in, view the dashboard, browse and filter the library, take a quiz, inspect feedback and history, complete a resource, and view recommendations. The Administrator can access the protected evaluation view. The external website is the destination represented by a resource URL; it is not hosted or controlled by this application.

FIGURE 4.3

Use-case view

Primary actors and their interactions with Dynamic Learning

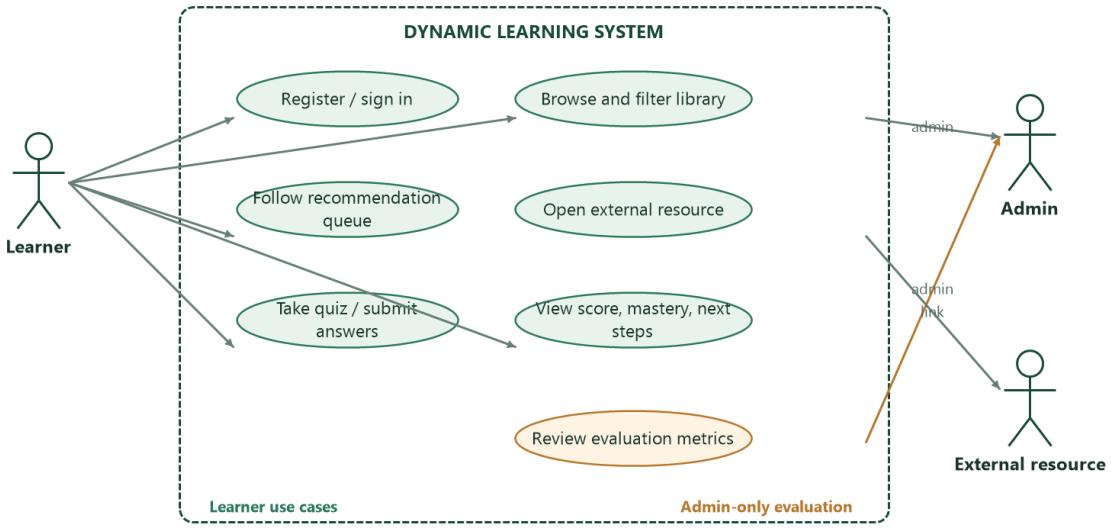


Figure 4.3: Use-case diagram

4.4 Class and data model

The SQLAlchemy data model represents the following entities and relationships:

- `User` stores an identifier, unique email, name, password hash, administrator flag, creation time, and relationships to attempts, completions, and mastery scores. Passwords are stored through a generated hash rather than as plain text.
- `Topic` stores a unique name and description and owns resources, quizzes, and prerequisite records.
- `Prerequisite` connects a target topic to a prerequisite topic, stores a minimum mastery value, and enforces uniqueness for a topic/prerequisite pair. Its model default is 0.6.
- `Resource` stores a topic, title, external URL, kind, difficulty, estimated minutes, and description.
- `Quiz` belongs to a topic and contains questions.
- `Question` stores a prompt, JSON options, and the answer used for server-side grading.
- `QuizAttempt` stores a user, quiz, score, JSON answers, and completion timestamp.
- `QuestionResponse` stores the attempt, question, submitted answer, and correctness value.
- `ResourceCompletion` stores a user-resource completion timestamp and enforces one row for each user-resource pair.
- `ResourceInteraction` stores a user, resource, event, and creation timestamp.
- `MasteryScore` stores a unique user-topic estimate and its update timestamp.

The model relationships make learner progress navigable from the user record while keeping raw attempts, responses, completion events, and estimates as distinct records. This is important because a mastery estimate is derived from evidence and should not replace the evidence history.

FIGURE 4.4

Core data model

SQLAlchemy entities, key attributes, and cardinalities used by the learning flow

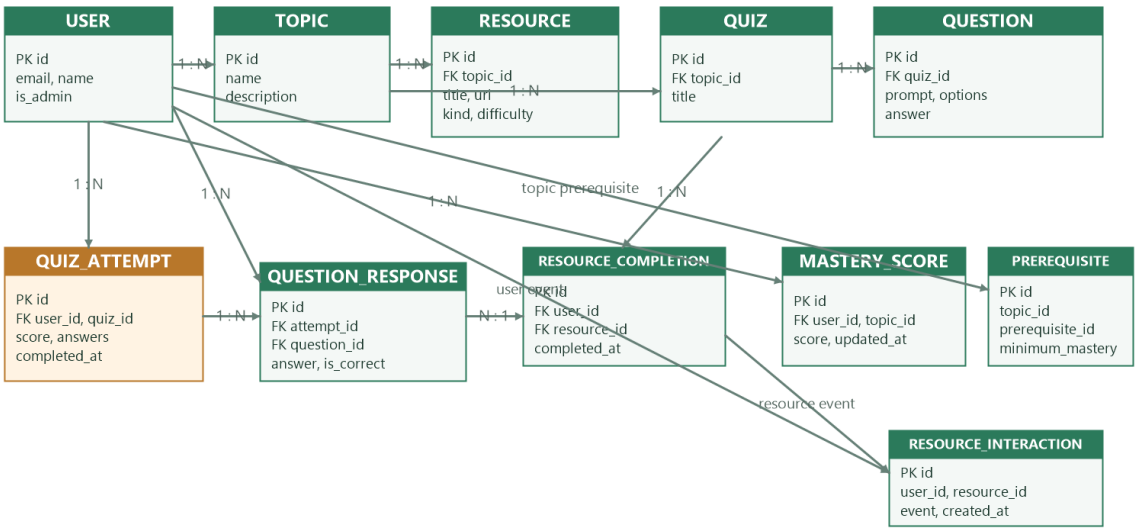


Figure 4.4: Class/data model diagram

4.5 Quiz-submission activity

The activity begins with quiz retrieval and answer entry. On submission, the route requires a JSON object and an `answers` object. It rejects an unknown question identifier, a non-string answer, or an answer that is not one of the question's options. Omitted answers are represented as empty answers and count as incorrect, preserving a complete grading pass over the quiz questions. Correctness is based on exact option equality.

The route calculates the score, creates the `QuizAttempt`, flushes it to obtain its identifier, creates `QuestionResponse` rows, calls the mastery update with `commit=False`, and commits the related changes together. If an exception occurs, the session is rolled back. The returned feedback contains per-question correctness, the submitted answer, correct answer, and explanation. The route also returns the previous and next recommendation snapshots and a boolean indicating whether recommendation signatures or the primary item changed.

FIGURE 4.5

Quiz submission activity

Validation, persistence, mastery update, and transparent recommendation feedback

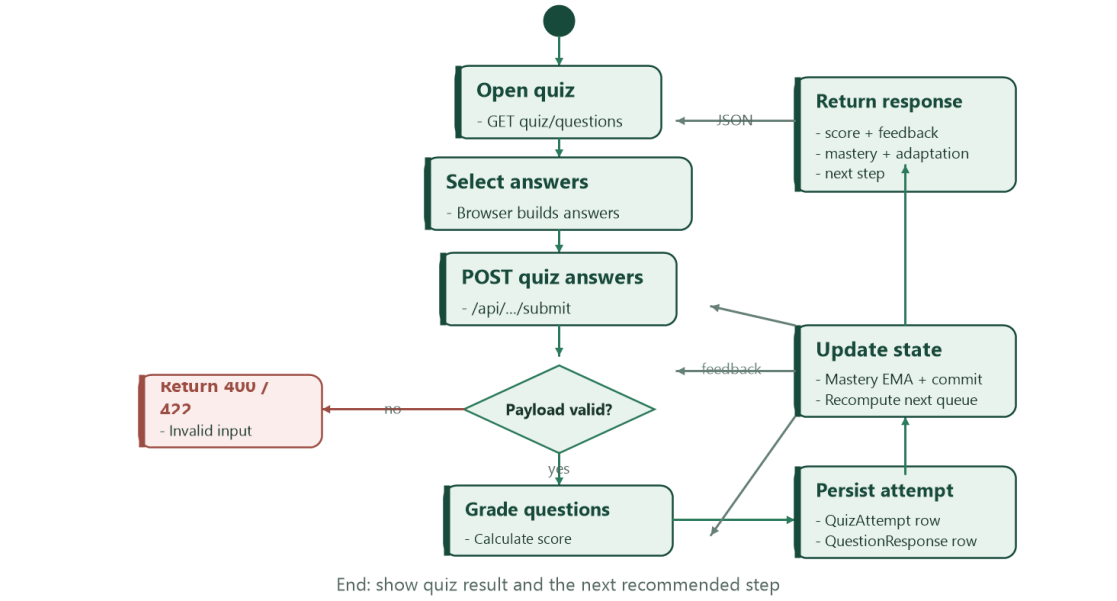


Figure 4.5: Quiz submission and adaptation activity diagram

4.6 Recommendation sequence

The browser first sends a request to a protected learning route. The route verifies the session and, for a JSON write, the CSRF boundary. The quiz payload is validated against the quiz's question identifiers and option arrays. The database stores the new attempt and responses, the mastery service refreshes the topic estimate, and the recommender recalculates the plan. The route serialises the plan as a primary item, supporting items, priorities, topics, and reasons. The library completion route similarly verifies the resource, preserves an existing completion, records a completion interaction, and returns a JSON status.

FIGURE 4.6

Quiz submission sequence

Browser, learning blueprint, services, and persistence collaborate to produce an adaptive response

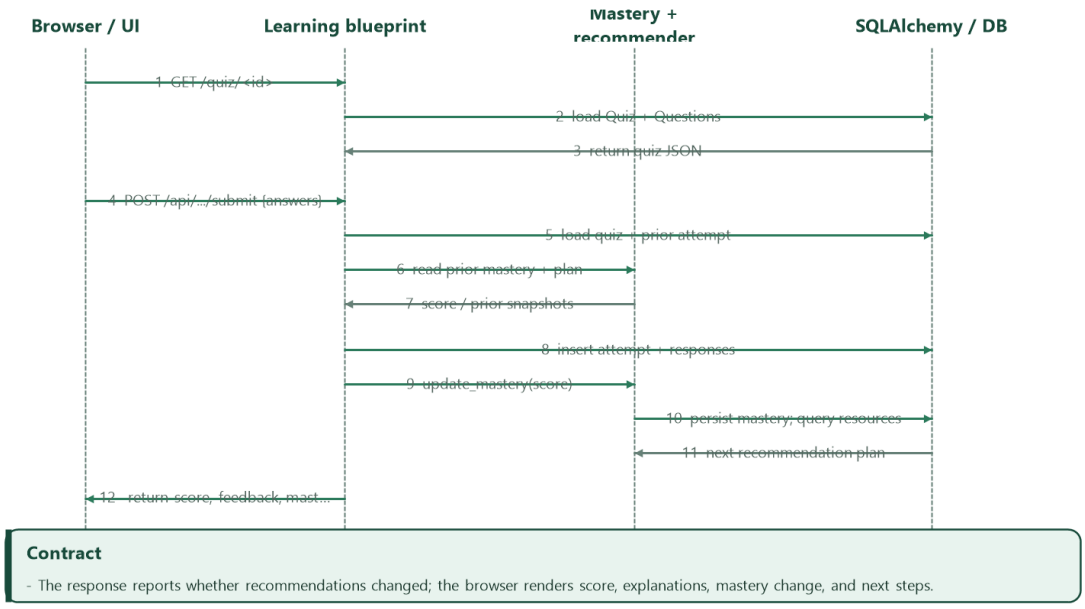


Figure 4.6: Recommendation-generation sequence diagram

Chapter 5 — Implementation

5.1 Modules and responsibilities

Table 5.1: Modules and responsibilities

Module	Responsibility
Authentication and access control	Registration, sign-in, session-bound access, and administrator-only evaluation access.
Catalog and learning library	Seeded topics and resources, browsing, external resource links, and text/topic/kind filtering.
Quiz and server-side scoring	Quiz retrieval, payload validation, exact-option grading, feedback, and response persistence.
Mastery and learner history	Topic-level estimate, ordered attempts, question responses, and displayed history.
Recommendation	Candidate filtering, prerequisite checks, priority scoring, deterministic ordering, and explanations.
Completion and revision	Unique resource completion, completion interactions, recent-completion suppression, and older revision eligibility.
Evaluation and baselines	Offline synthetic cases, adaptive/content/stable rankers, metrics, coverage, and ranking runtime.
Responsive interface	Jinja templates, CSS, progressive JavaScript enhancements, feedback, and accessible controls.

5.2 Data collection and server-side grading

The learning blueprint exposes authenticated endpoints for resources, the library, quizzes, completion, quiz retrieval, and quiz submission. `GET /api/resources` supports text, topic, kind, and limit handling over a recommendation-derived list. `GET /library` retrieves catalogue resources and passes the set of completion identifiers to the template so completion status survives refresh. `GET /quiz/<quiz_id>` renders the quiz page, while `GET /api/quizzes/<quiz_id>` returns question options and the topic's resource summaries.

The `POST /api/quizzes/<quiz_id>/submit` route is deliberately server-scored. The route first checks that the JSON body is an object and that `answers` is an object. It maps the quiz's questions by string identifier and rejects unknown question identifiers. Each supplied answer must be a string present in that question's option list. It then compares each answer exactly with the stored answer, computes the fraction correct, and records both the overall attempt and individual response rows. This prevents a client from supplying an arbitrary score as the learner evidence.

The returned adaptation structure includes the previous score where one exists, the latest score, score change, previous and latest mastery, mastery change, previous and next primary resources, previous and next step identifiers, and a `recommendations_changed` flag. A stable queue can therefore be reported as stable; a recommendation need not change solely because a quiz was submitted.

5.3 Mastery update

Mastery is a topic-level estimate derived from recorded quiz evidence. It is not a grade, directly observed knowledge, or a validated probability of answering a future item correctly. If evidence is sparse, noisy, or unrepresentative, the estimate can be wrong. Unassessed topics remain separate from low assessed estimates.

The current service uses the following EMA update, with the score and estimate bounded to the interval from 0 to 1:

```
new_mastery = old_mastery + 0.35 × (quiz_score − old_mastery)
```

The value 0.35 is a transparent starting heuristic. It gives the newest score a visible 35% influence while retaining 65% of the previous estimate. It was not learned from a learner study and is not presented as optimal or educationally validated. The audit further records that the manual sensitivity helper can replay custom rates, while the live attempt-replay path remains fixed at 0.35.

The application uses a 65% code constant to identify an assessed weak topic. This is a prototype threshold, not a validated cut point. A new learner with no attempt is therefore not automatically placed in the weak-topic list.

5.4 Candidate filtering and ranking

The recommendation service uses the learner's mastery, quiz evidence, prerequisite state, completion state, difficulty, and deterministic resource ordering. The documented priority formula is:

```
priority = 50 + 35 × (1 − mastery) − 3 × difficulty − 20 × unmet_prerequisite_count
```

Resources with recent completions are suppressed. A resource completed at least 14 days earlier can return as a revision candidate and can receive a time-policy explanation. The 14-day window is a transparent code constant, not a forgetting model. Revision eligibility does not add a validated revision priority boost. Difficulty is always penalised in the formula; the ranker does not switch to advanced-first behaviour after mastery. Latest low scores can affect explanations through recent evidence, but repeated mistakes and declining performance are not separate detectors in the current ranker. Ties are resolved by resource identifier so that the queue is deterministic.

The user interface presents one primary resource and up to two supporting resources. A reason may identify a low current mastery estimate, a low recent score, a prerequisite that should be studied first, or the time since a completion. These explanations describe the rule used by the prototype; they do not claim accurate prediction of forgetting or learning gain.

5.5 Completion and revision persistence

The `ResourceCompletion` model has a unique constraint on `(user_id, resource_id)`. The completion route checks for an existing row before adding a new one, and it records a `ResourceInteraction` with event `complete` for the request. Repeated completion requests therefore preserve one unique completed resource while the interaction history can still record the event. The library route passes completed resource identifiers so that the completion badge survives a refresh.

5.6 Explainability and UI contract

The dashboard receives the signed-in user, topics, assessed mastery mapping, and recommendation objects. The library receives resources, topics, filter values, and completion identifiers. The quiz page receives a quiz with an identifier, title, topic, questions, prompts, and options. The evaluation page receives observed metrics and the offline benchmark report. The interface is intentionally data-led: it does not invent progress, scores, or recommendations for a learner with no activity.

The README documents semantic landmarks, skip navigation, visible focus states, labelled form controls, meter elements for mastery, live regions, keyboard-selectable answer cards, reduced-motion support, and mobile breakpoints. JavaScript progressively enhances completion and quiz submission, while the normal form contract remains important where matching HTML responses exist. These are documented design properties; the audit did not use a browser rendering engine to certify them.

Chapter 6 — Testing

6.1 Verification strategy

Testing was performed at several levels: unit and integration checks through pytest, Python compilation, JavaScript syntax validation, dependency installation and consistency, security and access-control probes, a real HTTP workflow over a disposable disk SQLite database, and independent arithmetic checks for the offline evaluation. The purpose was to verify the local academic prototype and reproducibility boundaries, not to certify production operation or learning effectiveness.

The final fresh-environment run completed **167 passed in 32.99 seconds with `-W error`**, with no skipped tests or warnings in that run. An earlier baseline run reported **44 passed in 13.53 seconds** but missed several defects that the audit later exposed. The final audit also recorded:

- `python -m compileall -q app tests config.py seed.py run.py` completed successfully.
- `node --check app\static\js\app.js` completed successfully; this checks syntax and does not execute browser JavaScript.
- A fresh virtual environment installed `requirements.txt`, and `python -m pip check` returned `No broken requirements found.`
- Security checks covered password hashing, exact password preservation, malformed authentication and quiz input, session handling, missing and invalid CSRF tokens, HTML/API evaluation access, disabled demonstration logins and sessions, production safeguards, and reset guards.
- The HTTP workflow covered two seed runs, login, two quizzes, completion, refresh, new-user isolation, logout, process stop, a new application instance reading the same database, non-destructive seeding, and an isolated reset.
- Offline evaluation checks covered candidate-pool and K consistency, chronological pre-split filtering, label-output independence, metric arithmetic, coverage, runtime arithmetic, duplicate identifiers, short lists, absent labels, invalid K, and future-event exclusion.
- The mastery sensitivity check verified the three-score sequence at four rates and recorded actual primary/support identifiers and priority changes in an isolated manual-state replay.

6.2 Representative test plan and evidence

Table 6.1: Representative test plan and evidence

ID	Scenario	Expected result and recorded evidence
TC001	Register with valid details	Account creation and session access succeed in the HTTP workflow.
TC002	Login with valid credentials	The learner reaches the dashboard; the clean workflow recorded this path.
TC003	Missing or invalid CSRF	The write is rejected; the audit exercised missing and invalid tokens.
TC004	Submit exact quiz options	The server returns score and per-question feedback; the clean workflow recorded a 25% and then a 100% Variables quiz.
TC005	Malformed or unknown answer payload	The request is rejected safely rather than accepting an arbitrary answer structure.
TC006	Quiz updates mastery	Attempt, responses, and refreshed mastery commit together; a write-failure regression test verifies rollback.
TC007	Complete a resource twice	One unique completion remains and the

		completed badge survives refresh.
TC008	Revisit a recent completion	The recently completed resource is filtered from the immediate queue.
TC009	Anonymous, learner, and admin evaluation access	Anonymous receives 401, a learner receives 403, and an administrator receives 200.
TC010	Rerun normal seed	Learner records are preserved; the audit verified non-destructive repeated seeding.
TC011	Explicit development reset	An isolated SQLite database is cleared and the catalogue is reseeded; production and non-SQLite reset targets are refused.
TC012	Offline metric arithmetic	Precision@5, Recall@5, NDCG@5, coverage, and runtime calculations match their definitions.

The disk-database HTTP audit recorded a cold-start weak-topic list as empty, a Variables quiz score of 25% with mastery 0.0875, and a second Variables quiz score of 100% with previous score 25% and mastery 0.4069. It verified that reasons and priorities were reported as changed, although a changed explanation or priority does not necessarily mean that the resource identifier changed. Repeated completion requests left one unique completed resource, and the library badge survived refresh. A new learner could register but could not access evaluation; a `user_id` query parameter could not expose another user's private progress. After server shutdown, a new application read two attempts, eight responses, one completion, and the same mastery from disk. Ordinary seeding preserved those records. An explicit isolated reset restored seven topics, 21 resources, seven quizzes, and 25 questions together with the local development catalogue state.

6.3 Security and operational boundaries

Verified protections include Werkzeug password hashing, server-scored selected answers, session-bound private progress, Flask-WTF CSRF for form and JSON writes, HttpOnly/SameSite cookies, Secure cookies in production, and administrator-only evaluation routes. This was not a penetration test. Password-brute-force throttling, account recovery, multi-worker concurrency, TLS deployment, dependency vulnerability scanning, and long-running production operations were not certified.

Reset is a command-line action rather than a web endpoint. It checks the environment and database engines before dropping tables, refuses production/staging and non-SQLite targets, and remains destructive for development/test SQLite databases. It does not create an automatic backup. The application documentation therefore requires a verified backup before a reset of important data.

The audit records an important database side effect: an early probe mistakenly executed reset against the default workspace development database rather than an isolated database, and the catalogue was reseeded afterward. Any learner activity present before that reset may have been removed; no pre-reset backup or recovery was verified. This fact is disclosed here rather than described as preserved data. Later final workflow, security, reset, and persistence checks used in-memory or temporary databases outside the workspace.

6.4 Browser and packaging boundary

No browser rendering engine was used in the final audit. The HTTP workflow verifies routes, HTML rendering, session behaviour, and persistence, but it does not certify button-click JavaScript execution, responsive visual layout, keyboard behaviour, or visual accessibility. A manual browser walkthrough is recommended before submission. The original repository audit found no screenshot assets; the genuine

Dynamic Learning screenshots supplied afterward are included in Appendix 2 as packaging evidence, but they do not replace a fresh interactive browser validation.

Chapter 7 — Results and Discussion

7.1 Application results

The seeded catalogue contains seven Python topics, 21 resources, seven quizzes, and 25 questions. The project guide identifies Variables, Conditionals, Loops, and Functions as the working multi-question quiz topics. A clean workflow recorded quiz scores, topic mastery changes, recommendation explanations, recent-completion filtering, and persistence to a disposable disk database. The strict automated suite and the HTTP checks described in Chapter 6 passed in the final audit.

The application result is therefore functional and auditable behaviour. It demonstrates that the local flow can collect evidence, update an estimate, present a recommendation, and retain the state. It does not demonstrate that a learner learned more, that a particular learning rate is optimal, or that the queue predicts future success.

7.2 Exact synthetic offline benchmark

The regenerated offline dataset is named `synthetic-learning-scenarios-v2`. It contains 21 resources, four cases, four labelled cases, one case per scenario, and a cutoff of K=5. The cases are named beginner cold start, struggling with a topic, ready for advanced material, and revision due. The last two names require particular care: “ready for advanced material” is an author-assigned scenario name even though the replayed mastery values remain below the application's thresholds, and “revision due” is a time-policy case without retention evidence.

The exact aggregate benchmark table reported by the audit is:

Table 7.1: Synthetic offline benchmark results

Method	Precision@5	Recall@5	NDCG@5	Coverage	Illustrative mean ranking runtime
Adaptive	0.050000	0.250000	0.250000	0.285714 (6/21)	about 0.029 ms
Stable catalog baseline	0.150000	0.500000	0.375316	0.238095 (5/21)	about 0.003 ms
Content-based baseline	0.250000	1.000000	1.000000	0.571429 (12/21)	about 0.007 ms

On these four synthetic cases, the adaptive ranker is below both baselines on the aggregate Precision@5, Recall@5, and NDCG@5 values. This is a result about this benchmark and its assumptions, not a failure claim about the entire application. The comparison used an identical candidate tuple and K for all three methods, but identical candidate pools do not imply identical information: the stable method ignores history, the adaptive method uses history, and the content-based method receives explicit target-topic and difficulty features authored alongside the labels.

The per-scenario values below are reported as **Precision / Recall / NDCG at K=5**:

Table 7.2: Per-scenario synthetic results

Scenario	Adaptive	Stable	Content-based
Beginner cold start	0.2 / 1 / 1	0.2 / 1 / 1	0.2 / 1 / 1
Struggling with a topic	0 / 0 / 0	0.4 / 1 / 0.501266	0.4 / 1 / 1
Ready for advanced material (author-assigned name)	0 / 0 / 0	0 / 0 / 0	0.2 / 1 / 1
Revision due (time-policy	0 / 0 / 0	0 / 0 / 0	0.2 / 1 / 1

case)			

7.3 Benchmark metric definitions and validity limits

Precision@K is the number of relevant hits divided by K, including when fewer than K items are relevant. Recall@K is the number of relevant hits divided by the number of labelled relevant resources. Binary NDCG@K is the discounted gain divided by ideal discounted gain. A case with no labels returns `null` and is excluded from precision, recall, and NDCG macro averages. Coverage is the union of distinct returned top-K resource identifiers divided by the full catalogue size. Runtime is measured with `perf_counter()` around one ranking call per method and case; it excludes catalogue loading, shared filtering, metrics, HTTP, database, network, and rendering costs. The values in the table are therefore not throughput, load, or end-to-end latency evidence.

The labels are independent of ranking outputs in the limited sense verified by reversing the rankers without changing the generated labels. They are not independent human or learner judgments. The topic and difficulty metadata overlap with the authored relevance rules, and the content-based method receives information aligned with those rules. There is one hand-built case per scenario, the scenario is confounded with selected topic and catalogue order, and there are no repetitions, balanced topic coverage, confidence intervals, significance tests, external annotators, inter-rater agreement, or held-out real-world validation. K=5 is larger than the one or two labelled items in some cases, so perfect recall need not imply high precision.

The advanced-ready history is not evidence of demonstrated readiness. Two topic scores of 0.9 and 0.95 replay to an EMA value of 0.5373, below the project's 0.65 weak-topic threshold, and the prerequisite histories yield 0.315, below 0.6. The scenario name expresses author intent, not a fact established by the application. Similarly, the revision scenario uses a 21-day-old completion outside the 14-day filter window, but has no quiz or retention evidence. The adaptive ranker allows the older resource back into the candidate pool; it does not currently add a validated revision boost.

The benchmark is thus useful as a reproducible audit of small deterministic rankers, metric arithmetic, candidate consistency, and assumptions. It is not evidence of superiority, novelty, statistical significance, or educational improvement.

7.4 Learning-rate sensitivity

The sensitivity check freezes the seeded catalogue, initial estimate of 0.0, prerequisite data, completion state, candidate pool, and the ordered synthetic score sequence `[0.8, 0.8, 0.8]`. It uses the manual no-attempt-row branch for rates 0.20, 0.35, 0.50, and 0.80. Every update is rounded to four decimals. The live replay of recorded quiz attempts remains fixed at 0.35; the custom manual argument is not a global setting.

The formula results are:

Table 7.3: Learning-rate sensitivity formula results

Rate	Weight on newest score	After one 0.8 score	After three 0.8 scores	Likely queue sensitivity
0.20	20%	0.1600	0.3904	A previously weak topic stays weak longer, so its resources can remain primary after a good attempt.
0.35	35%	0.2800	0.5803	Balanced starting behaviour; prerequisite

				and difficulty rules can still determine the primary item.
0.50	50%	0.4000	0.7000	A few good scores can move a topic above a threshold sooner, allowing another topic to become primary.
0.80	80%	0.6400	0.7936	One new score can reorder the queue quickly; a single noisy result has more impact.

The actual seeded manual-state replay recorded the following primary and supporting identifiers and Variables beginner priority values:

Table 7.4: Learning-rate sensitivity replay

Alpha	After one	After three	Actual primary + support IDs in seeded manual-state replay	Variables beginner priority
0.20	0.1600	0.3904	1, 2, 4	68.34
0.35	0.2800	0.5803	4, 16, 1	61.69
0.50	0.4000	0.7000	4, 5, 7	57.50
0.80	0.6400	0.7936	4, 5, 7	54.22

The identifiers are tied to the fresh seed order: 1 and 2 are Variables guide/practice, 4 and 5 are Data Types guide/practice, 7 is the Conditionals guide, and 16 is the Lists guide. The numbers show that the queue can change as the estimate changes, but prerequisite penalties, difficulty, completion filtering, and deterministic tie-breaking can outweigh the mastery change. This is a mathematical and manual-state sensitivity check, not a tuning experiment, a learner study, or evidence that 0.35 is best.

Chapter 8 — User Manual

8.1 Local setup

1. Create and activate a Python virtual environment.
2. Install the dependencies from `requirements.txt`.
3. Set process environment variables as needed. In particular, use a long random `SECRET_KEY` for an appropriate deployment environment. The application does not automatically load a `.env` file.
4. Run `python seed.py` to create the demonstration catalogue in a fresh development database.
5. Start local development with `flask --app run.py run --debug`.
6. Open `/register` and create a learner account.
7. Sign in and inspect the dashboard.

The seed catalogue supplies seven topics, 21 resources, seven quizzes, and foundation questions. Development-only demonstration-account details are intentionally not reproduced in this report. A production deployment must not rely on a published development credential, must use an appropriate random secret, and must disable the demonstration account.

8.2 Learner workflow

1. On the first dashboard visit, observe the cold-start state. Topics without quiz evidence should appear as **Not assessed**, rather than as weak scores.
2. Open the learning library and filter resources by search text, topic, or kind.
3. Open a foundation quiz for Variables, Conditionals, Loops, or Functions.
4. Submit answers and inspect the server-calculated score, question feedback, updated mastery estimate, and recommendation adaptation information.
5. Return to the dashboard and review the topic-wise history and recommendation reasons.
6. Open a learning resource from the library and mark it complete.
7. Refresh the library or dashboard to verify the completion state and observe that a recent completion is not immediately returned as a recommendation.
8. Continue with another quiz or resource. An older completion may become eligible under the revision time policy; this policy does not prove forgetting.

8.3 Administrator evaluation

The `/evaluation` and `/api/evaluation` routes are administrator-only. Evaluation access should be tested with a separately managed local administrator account. The view reports observed application activity separately from the offline synthetic benchmark. It should be interpreted using the assumptions and validity limits in Chapter 7.

8.4 Troubleshooting

- **`TemplateNotFound`:** run the application from the repository root and confirm that templates are under `app/templates/` and assets are under `app/static/`.
- **A page redirects to sign in:** the learner session is absent or expired; sign in again and retain the session cookie for API requests.
- **CSRF error:** keep the hidden `csrf_token` field for forms and send the `X-CSRFToken` header for JSON enhancements. `/api/csrf` is available for API clients.
- **Empty dashboard:** run `python seed.py` and complete at least one quiz. A new user intentionally starts with a cold-start catalogue order.

- **Clean demonstration state:** stop the development server and use `python seed.py --reset` only for disposable development/test SQLite data after checking the warning and ensuring a verified backup where appropriate. The reset is destructive.
- **Evaluation forbidden:** use an administrator account; do not remove the access check.
- **Recommendations do not change:** verify that answers use question identifiers and that completion requests use the intended resource identifier. The test suite covers mastery updates and completed-resource filtering.
- **Database migration issue:** the current app creates tables automatically for its small development schema. A production deployment should use an explicit migration tool and a managed database URL.

Chapter 9 — Conclusion

Dynamic Learning delivers a working local Flask prototype for explainable learning-resource recommendation. It persists users, quiz attempts, question responses, resource completions, resource interactions, prerequisites, and topic-level mastery estimates. It provides a seven-topic Python catalogue, working foundation quizzes, a learning library, a dashboard, a deterministic recommendation queue, and human-readable explanations. The learning route validates submitted answers on the server, grades exact options, commits attempt/response/mastery state together, and reports how the recommendation plan changed or remained stable.

The final audit provides evidence of functional correctness and reproducibility for the local academic prototype: 167 strict-suite tests passed, compilation and JavaScript syntax checks passed, dependencies were checked, security boundaries were exercised, and a disposable HTTP workflow verified persistence and reset behaviour. The exact synthetic benchmark is also transparent: on its four hand-built cases, the adaptive ranker scored below the stable and content-based baselines on the aggregate ranking metrics. That result, together with the benchmark's authored labels and feature overlap, must be reported rather than hidden.

The project therefore demonstrates implementation, inspectability, and auditability. It does not establish verified learning improvement, recommender superiority, production readiness, novel research, or accurate forgetting prediction. Mastery is an estimate, the 65% threshold and 14-day revision window are unvalidated constants, and the 0.35 learning rate is a starting heuristic. A browser walkthrough and a real learner study remain outside the evidence supplied by the audit.

Chapter 10 — Future Enhancement

The next research and engineering phase should address the evidence limitations directly:

1. Collect anonymized interaction data only with appropriate consent and ethics approval.
2. Add relevance labels reviewed independently of the ranker's feature inputs and hold them out chronologically for evaluation.
3. Expand the benchmark with repeated, balanced scenarios across topics, rather than one hand-built case per scenario.
4. Compare the rule-based method with the included stable and content-based baselines under comparable information conditions, and add a collaborative-filtering baseline only when suitable interaction data is available.
5. Validate the mastery threshold, learning-rate choice, and revision timing with pre-test, intervention, post-test, and delayed retention measures. Clicks and completions alone are not learning outcomes.
6. Add instructor tools for editing resources, questions, and prerequisite relationships.
7. Add explicit database migrations, deployment observability, stronger operational controls, and broader browser and accessibility testing.
8. Obtain readable full texts and passage-level citations before presenting any proposed literature gap as a finding.

These enhancements are future work, not capabilities claimed for the current implementation.

APPENDIX 1 — SUPPLIED BASE PAPER

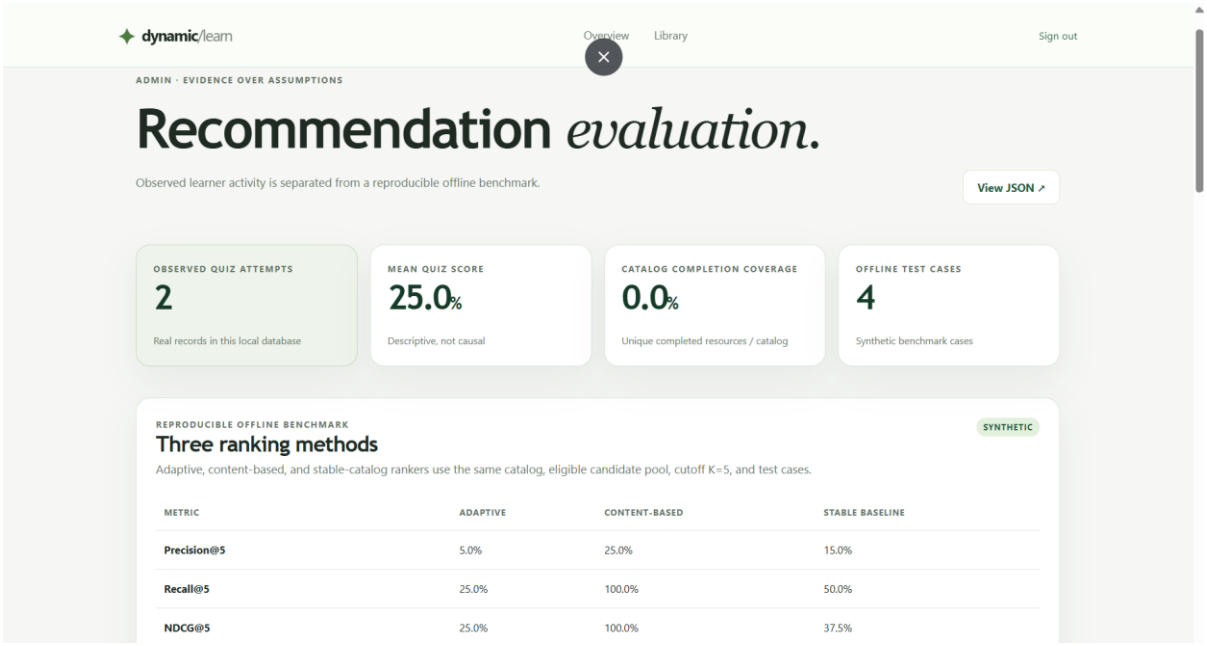
The supplied base paper for this project is Hongnan You, Lijun Xu, and Dongping Wu, "MFF-CF: A Multi-Feature Fused Collaborative Filtering Framework for Personalized Learning Resource Recommendation in Smart Education," Discover Artificial Intelligence, volume 6, article 860, 2026, DOI: 10.1007/s44163-026-01715-0.

Crossref metadata verified the publication identity, authors, journal, DOI, and online date of 21 August 2026. The full Springer article was not readable during the project audit because the publisher returned a client challenge. Dynamic Learning is not a reproduction of MFF-CF and does not implement collaborative filtering. This appendix therefore records the verified citation and project relationship rather than reproducing paper pages that were not supplied in readable form.

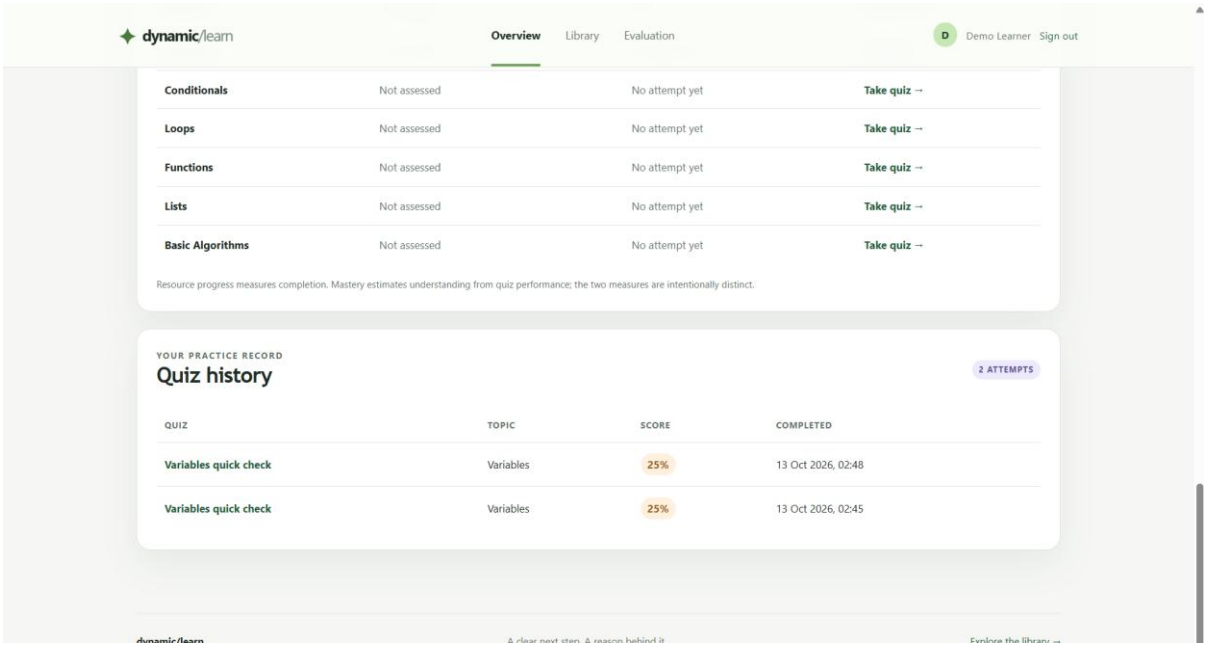
The project instead investigates a narrower implementation question: how topic-level quiz evidence, explicit prerequisites, completion and revision rules, and recommendation reasons can be made visible and reproducible in a small local learner flow when collaborative interaction data is unavailable.

APPENDIX 2 — SCREENSHOTS

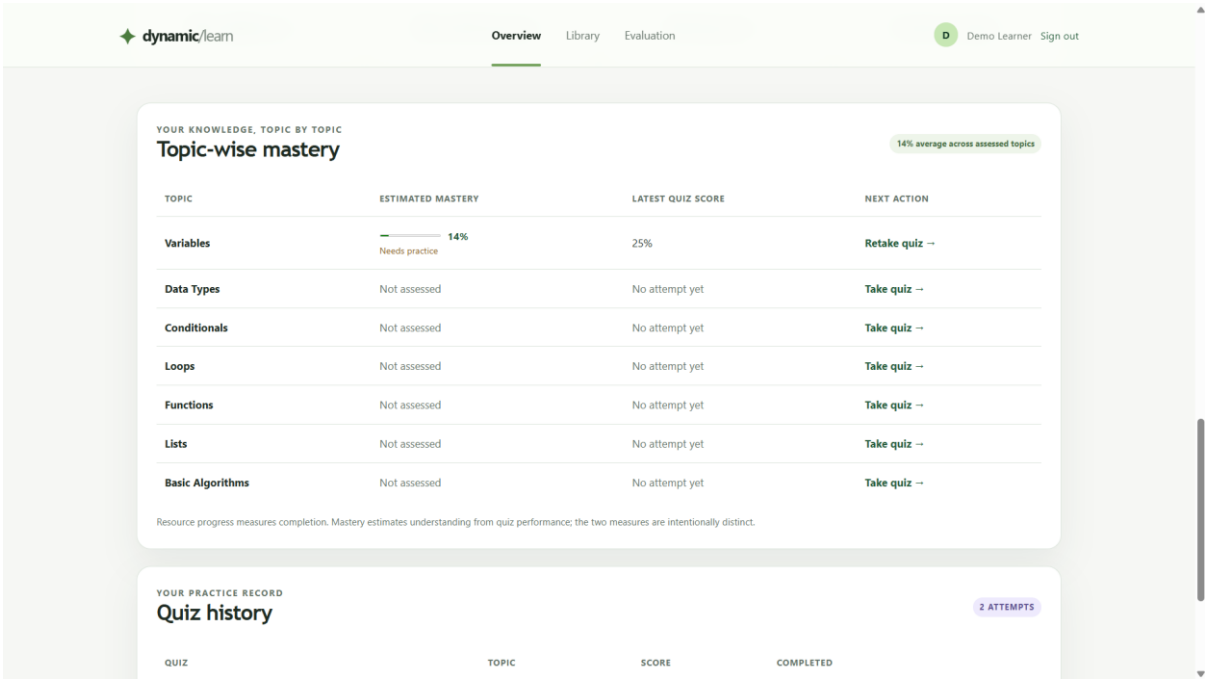
The following screenshots were supplied from the Dynamic Learning application and are included as visual evidence of the interface. They are not substituted with screenshots from the unrelated senior report. The captures represent different local states; some show completed badges while another dashboard capture reports zero overall completion. This apparent state difference is retained as supplied and is not used as a new functional-test claim.



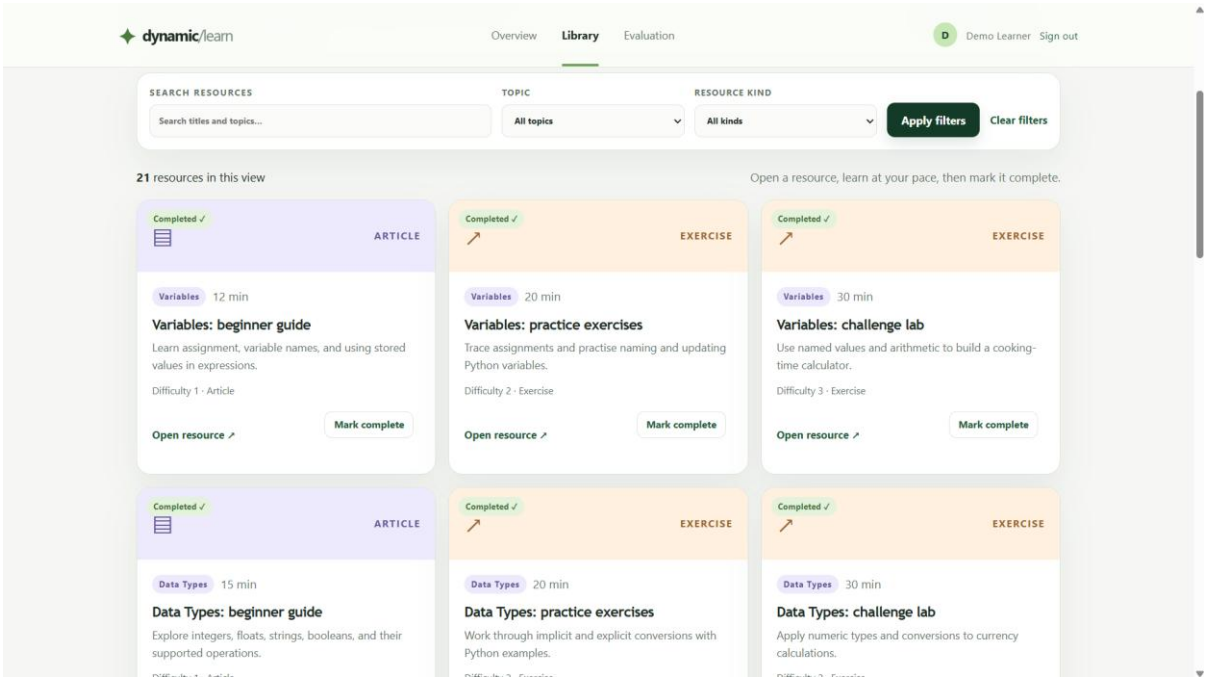
Screenshot 1: Recommendation evaluation benchmark



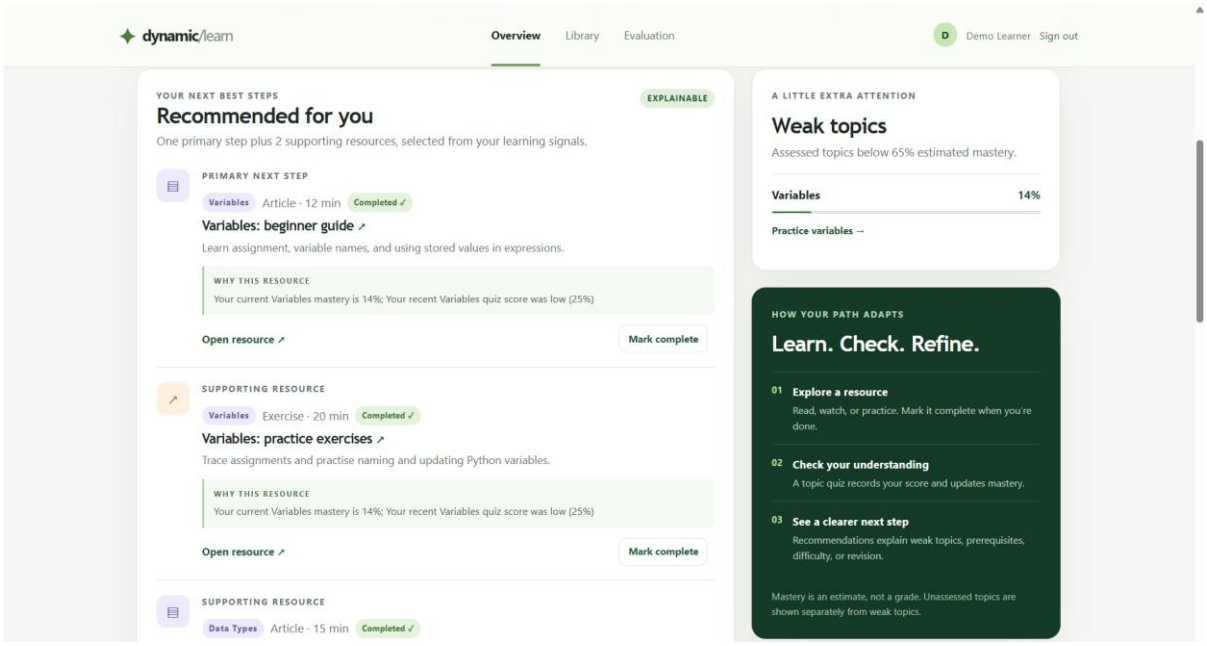
Screenshot 2: Dashboard quiz history



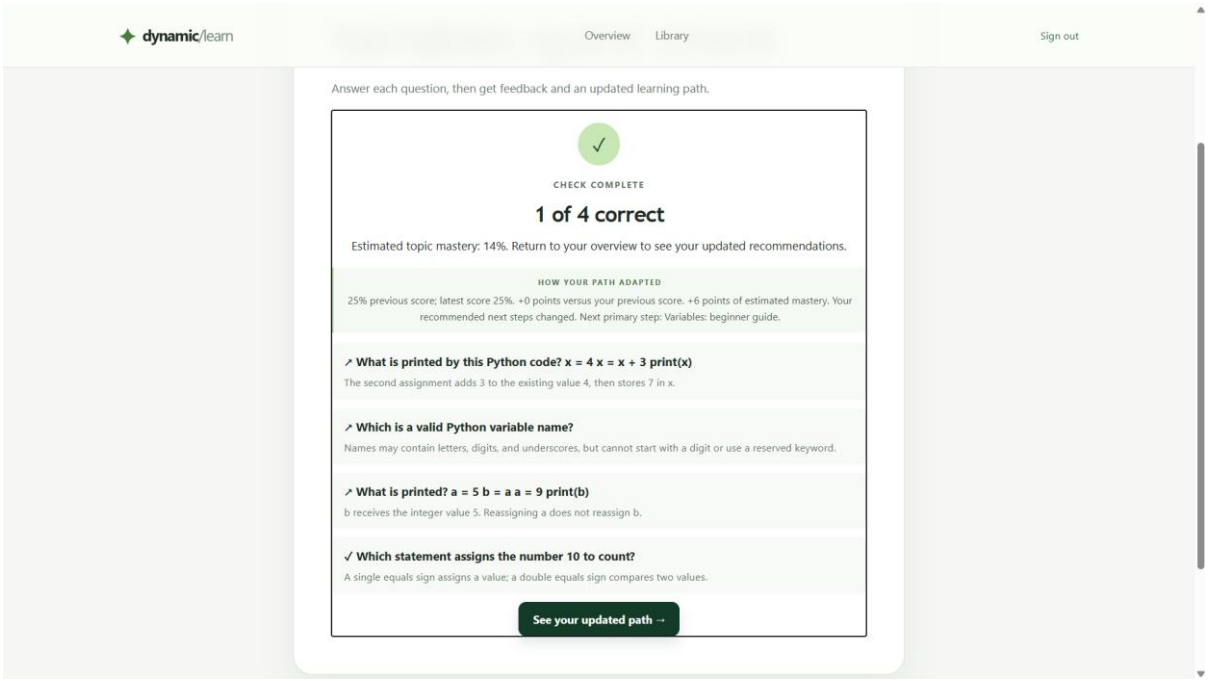
Screenshot 3: Dashboard topic-wise mastery



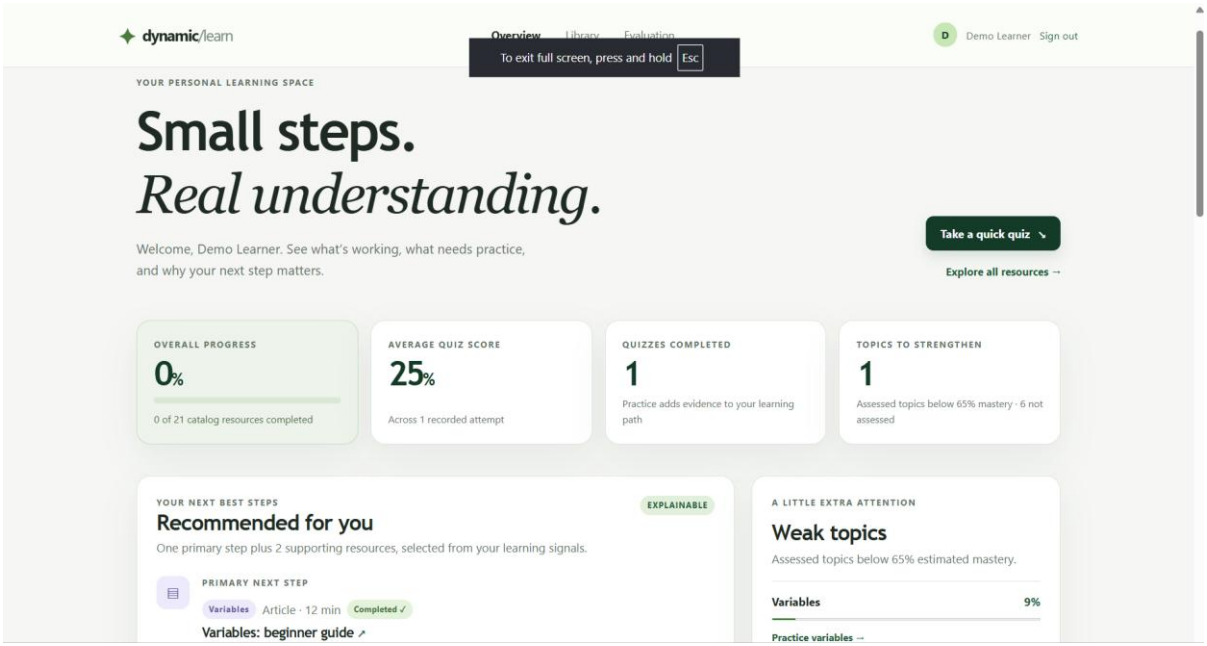
Screenshot 4: Learning resource library



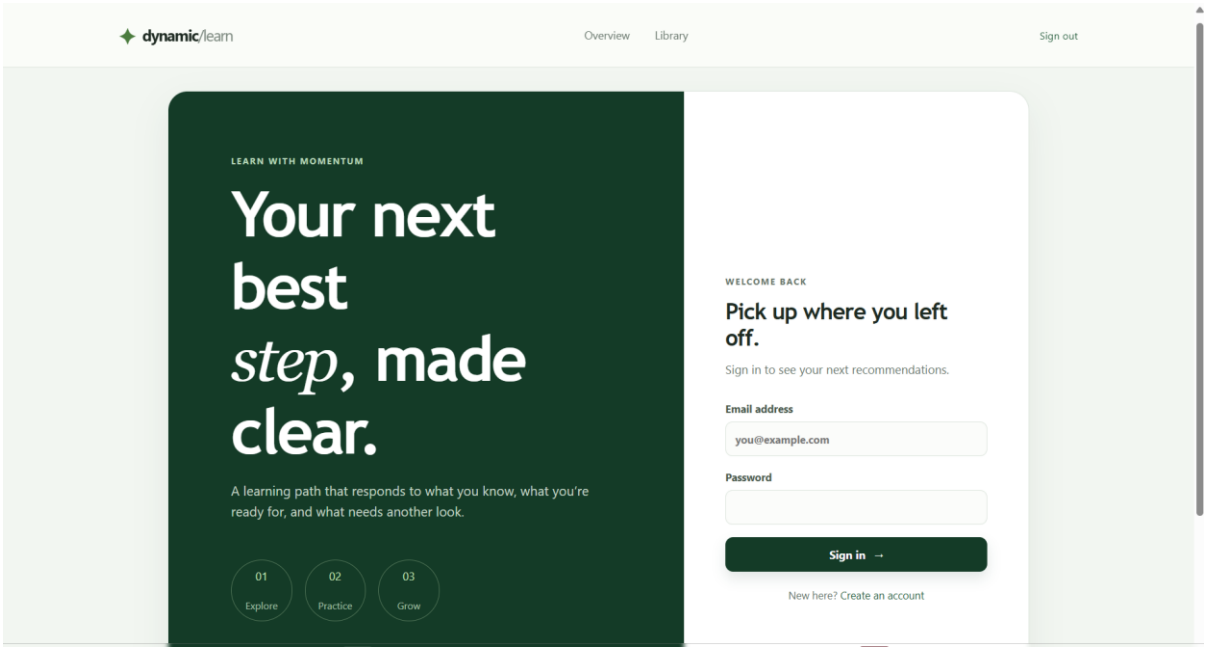
Screenshot 5: Dashboard recommendations and weak topic



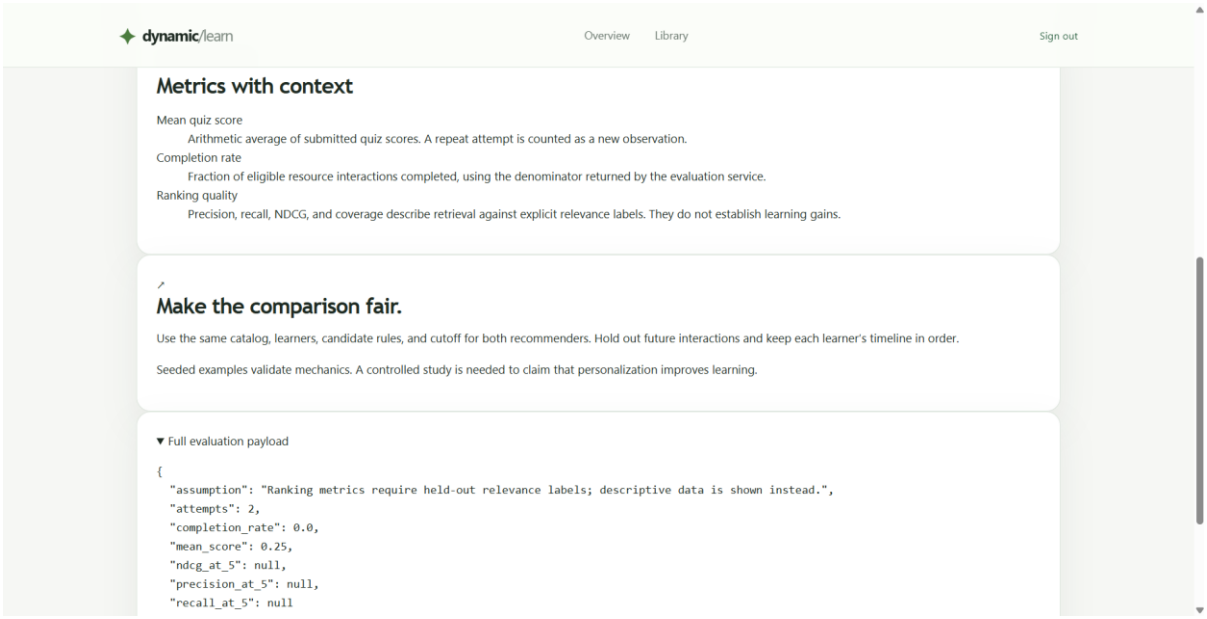
Screenshot 6: Quiz feedback and adaptation summary



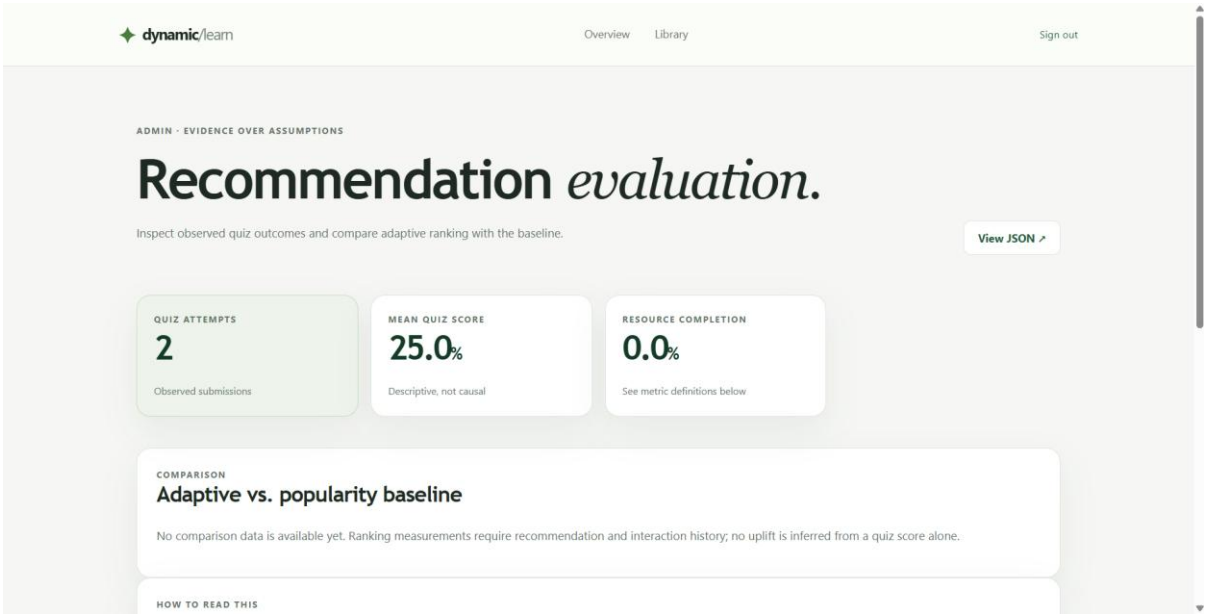
Screenshot 7: Dashboard overview and recommendation queue



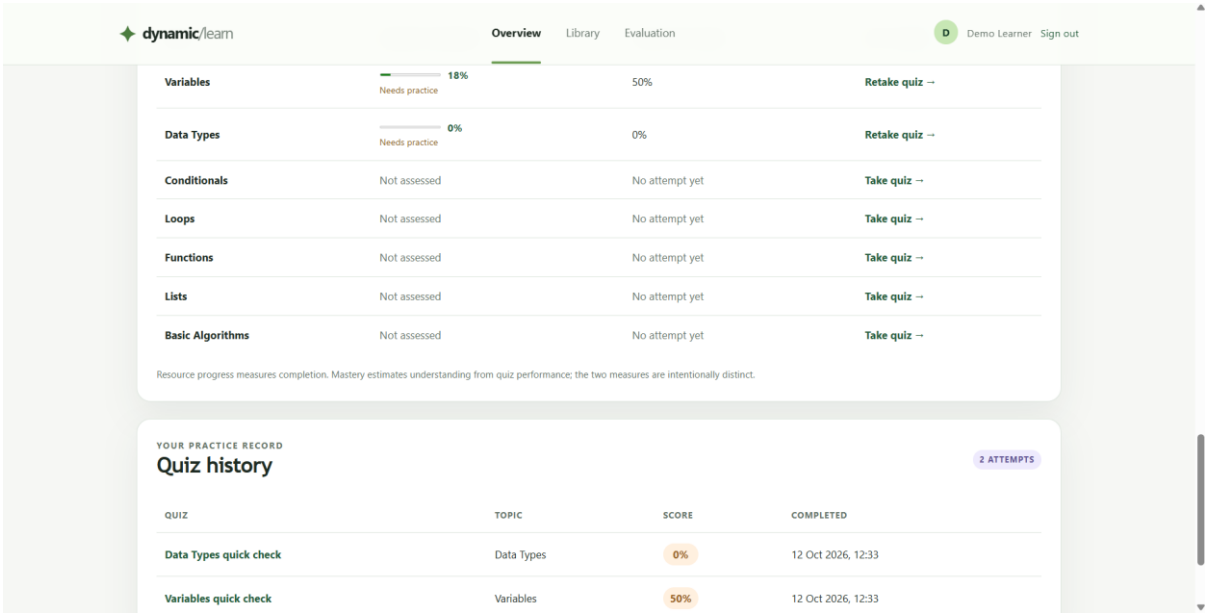
Screenshot 8: Sign-in page



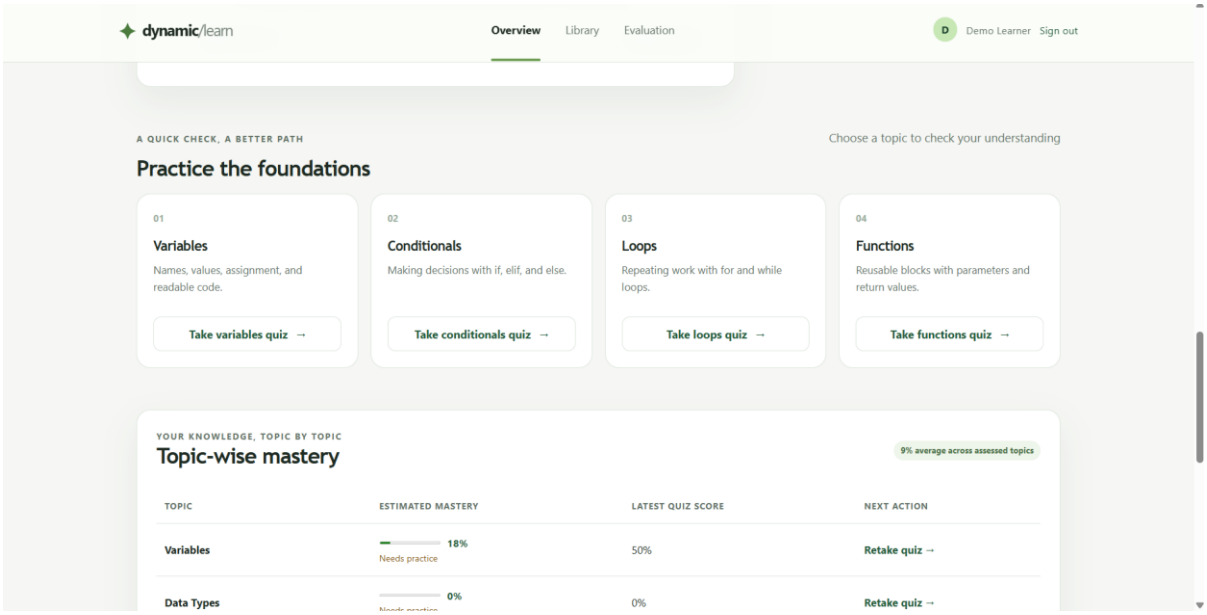
Screenshot 9: Evaluation metrics with context



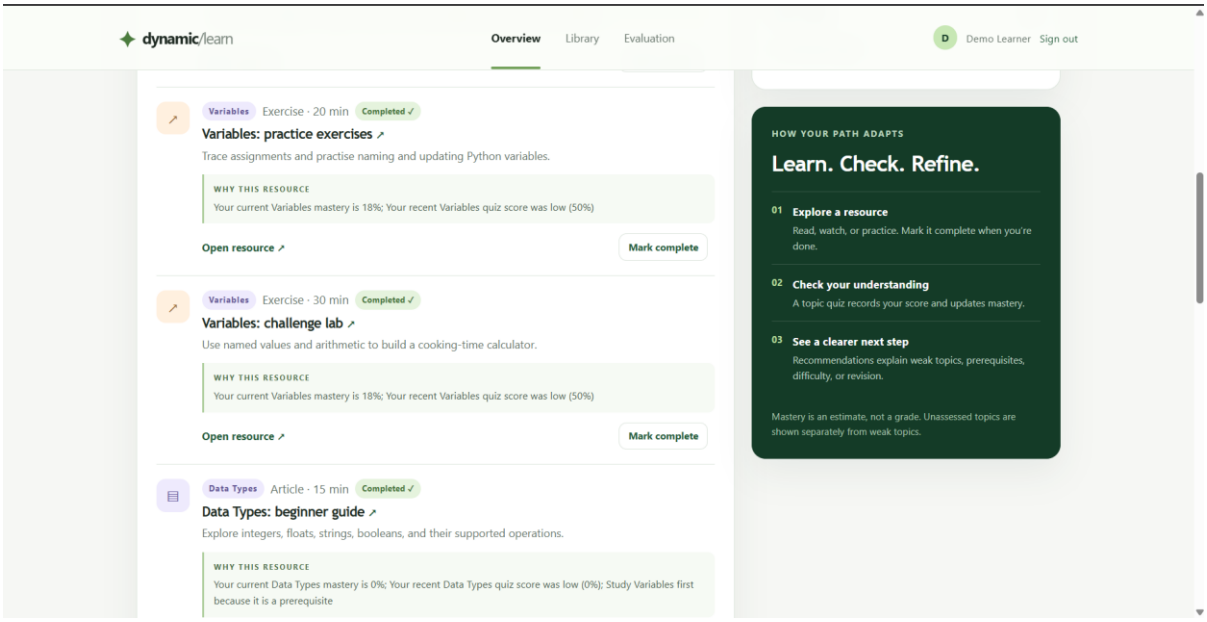
Screenshot 10: Evaluation comparison with no comparison data



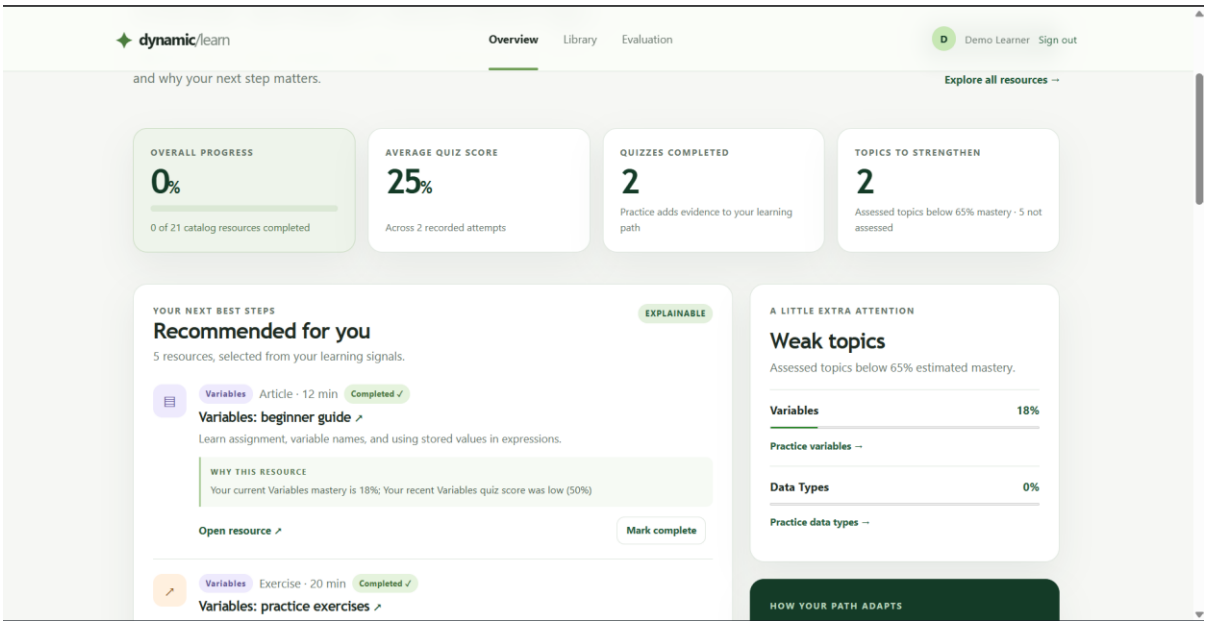
Screenshot 11: Topic-wise mastery and quiz history



Screenshot 12: Foundation quiz selection



Screenshot 13: Recommendation explanations and library state



Screenshot 14: Dashboard metrics and recommendation queue

APPENDIX 3 — INTERNSHIP CERTIFICATE

The supplied DVein Innovations internship completion certificate is reproduced below unchanged. It records a Full Stack with Java internship from 02/06/2026 to 02/07/2026. It is included as supporting academic material and is not evidence of Dynamic Learning ranking or learning outcomes.



CERTIFICATE

of Completion

This Certificate is Presented to

Mr. Novin kumar M

Anand Institute of Higher Technology

who has successfully completed the Full Stack with Java intership with DVein Innovations Pvt Ltd from (02/06/2026) to (02/07/2026). During this period, the intern worked on Software Projects and successfully presented Project outcome We wish the intern continued success in future endeavours

Sahana R
Executive Head

Selvamani S
Mentor

APPENDIX 4 — EVIDENCE AND DISCLOSURE NOTE

The report distinguishes application evidence from research claims. Application evidence includes deterministic local behaviour, stored learner records, automated tests, HTTP persistence checks, and synthetic ranking calculations. The audit does not certify a superior recommender, unbiased effectiveness, novel research, real learning improvement, accurate forgetting detection, full-paper limitation claims, real-browser execution, production load/concurrency, or preservation and recovery of learner data that may have existed before the early workspace reset probe.

REFERENCES

[1] P. Brusilovsky and E. Millán, “User Models for Adaptive Hypermedia and Adaptive Educational Systems,” in *The Adaptive Web*, pp. 3–53, 2007, DOI: 10.1007/978-3-540-72079-9_1. The publication identity and pages were verified through DOI/Crossref; the full Springer text was blocked during the project audit.

[2] N. Manouselis, H. Drachsler, R. Verbert, and E. Duval, *Recommender Systems for Learning*, SpringerBriefs, 2013, DOI: 10.1007/978-1-4614-4361-2. The matching book and print year were verified through Crossref; it is distinct from a different 2011 TEL chapter, and full-text limitation claims were not passage-verified.

[3] C. Piech, M. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, “Deep Knowledge Tracing,” in *Advances in Neural Information Processing Systems 28*, 2015. Proceedings record: https://papers.nips.cc/paper/5654-deep-knowledge-tracing. The proceedings abstract was readable during the audit and describes recurrent neural networks for modelling student interactions and predictive evaluation on real student data.

[4] B. Settles and M. Meeder, “A Trainable Spaced Repetition Model for Language Learning,” in *Proceedings of ACL*, pp. 1848–1858, 2016, DOI: 10.18653/v1/P16-1174. Record: https://aclanthology.org/P16-1174/. The identity, venue, year, and pages were verified; the PDF extraction used in the audit was not readable.

[5] H. You, L. Xu, and D. Wu, “MFF-CF: A Multi-Feature Fused Collaborative Filtering Framework for Personalized Learning Resource Recommendation in Smart Education,” *Discover Artificial Intelligence*, vol. 6, article 860, 2026, DOI: 10.1007/s44163-026-01715-0. Crossref record: https://api.crossref.org/works/10.1007/s44163-026-01715-0. Metadata verified the identity and 21 August 2026 online date; the full Springer article was not readable during the audit.

[6] *Dynamic Learning*, “README.md,” project implementation and reproducibility documentation, October 2026.

[7] *Dynamic Learning*, “PROJECT_GUIDE.md,” project guide, verified literature map, objectives, algorithm description, limitations, and future work, October 2026.

[8] *Dynamic Learning*, “SUBMISSION_AUDIT.md,” submission-readiness audit and regenerated synthetic evaluation evidence, 12 October 2026.

[9] *Dynamic Learning*, `app/models.py`, SQLAlchemy persistence models for users, catalogue entities, learner evidence, completions, interactions, and mastery scores.

[10] *Dynamic Learning*, `app/routes/learning.py`, authenticated learning routes, resource completion, quiz retrieval, server-side grading, atomic mastery update, and adaptation response implementation.